

Automated Metaheuristic Algorithm Design With Autoregressive Learning

Qi Zhao^{1b}, Tengfei Liu^{1b}, Bai Yan^{1b}, Qiqi Duan^{1b}, Jian Yang, and Yuhui Shi^{1b}, *Fellow, IEEE*

Abstract—Automated design of metaheuristic algorithms offers an attractive avenue to reduce human effort and gain enhanced performance beyond human intuition. Current automated methods design algorithms within a *fixed* structure and operate from *scratch*. This poses a clear gap toward fully discovering potentials over the metaheuristic family and fertilizing from prior design experience. To bridge the gap, this article proposes an autoregressive learning-based designer for automated design of metaheuristic algorithms. Our designer formulates metaheuristic algorithm design as a sequence generation task, and harnesses an autoregressive generative network to handle the task. This offers two advances. First, through autoregressive inference, the designer generates algorithms with diverse lengths and structures, enabling to fully discover potentials over the metaheuristic family. Second, prior design knowledge learned and accumulated in neurons of the designer can be retrieved for designing algorithms for future problems, paving the way to continual design of algorithms for open-ended problem solving. Extensive experiments on numeral benchmarks and real-world problems reveal that the proposed designer generates algorithms that outperform all human-created baselines on 24 out of 25 test problems. The generated algorithms display various structures and behaviors, reasonably fitting for different problem-solving contexts. Code is available at <https://github.com/auto4opt/ALDes>.

Index Terms—Automated design, evolutionary algorithm, learning, metaheuristic, optimization, transformer.

Received 5 May 2024; revised 29 July 2024; accepted 9 September 2024. Date of publication 20 September 2024; date of current version 8 October 2025. This work was supported in part by the National Science Foundation of China under Grant 72401122 and Grant 61761136008; in part by the Guangdong Basic and Applied Basic Research Foundation under Grant 2024A1515012241 and Grant 2021A1515110024; in part by the Shenzhen Fundamental Research Program under Grant JCYJ20200109141235597; in part by the Shenzhen Peacock Plan under Grant KQTD2016112514355531; and in part by the Program for Guangdong Introducing Innovative and Entrepreneurial Teams under Grant 2017ZT07X386. This article was approved by Associate Editor C. Doerr. (*Corresponding author: Yuhui Shi.*)

Qi Zhao, Tengfei Liu, Qiqi Duan, Jian Yang, and Yuhui Shi are with the Department of Computer Science and Engineering, Southern University of Science and Technology, Shenzhen 518055, China (e-mail: zhaohq@sustech.edu.cn; 12332470@mail.sustech.edu.cn; 11749325@mail.sustech.edu.cn; yangj33@sustech.edu.cn; shiyh@sustech.edu.cn).

Bai Yan is with the Research Institute of Trustworthy Autonomous Systems, Southern University of Science and Technology, Shenzhen 518055, China (e-mail: yanb@sustech.edu.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TEVC.2024.3464677>, provided by the authors.

Digital Object Identifier 10.1109/TEVC.2024.3464677

I. INTRODUCTION

METAHEURISTIC is a stochastic search methodology that integrates gradient-free local improvement with high-level strategies of escaping from local optima [1]. It involves a rich family of algorithms ranging from neighborhood search-based to evolutionary and swarm ones. These algorithms can be used for any problem with an available solution representation, solution quality evaluation, and notion of locality¹ [2]. This has made metaheuristic algorithms dominant in solving hard problems that do not meet analytical algorithms' rigorous mathematical assumptions, e.g., convexity and differentiability.

Given a target problem, carefully designing a metaheuristic algorithm over the rich choices of search components, algorithmic structures, and hyperparameters is necessary, in order to obtain good enough solutions. The design is usually human-made, criticized as laborious, untraceable regarding what motivate certain design decisions, and susceptible to accidental human bias, let alone that human designers are not always available [3]. This motivates the automated design of metaheuristic algorithms [3], [4], [5].

The automated design could find either instantiations/variants of existing algorithms or unseen algorithms to fit for target problem solving. This is different with automated algorithm selection [6] and automated hyperparameter configuration [7]. The former selects off-the-peg algorithms from a portfolio; the latter configures hyperparameters of a given algorithm. The automated design can be conducted either *offline* by a target distribution of problem instances or *online* by the search trajectory [2]. This article focuses on *offline automated design*, which suits scenarios where one can afford a priori computational resource (for algorithm design) to subsequently solve many problem instances drawn from the target domain.

Several methods have been introduced to offline automated design of metaheuristic algorithms. Representatives include SMAC [8], [9], irace [10], ParamILS [11], and offline hyperheuristics [12], [13]. SMAC uses Bayesian optimization to learn a surrogate function that models the mapping from algorithms to their performance on a target problem. Irace proposes iterative racing to select desired algorithms for a target problem, subsequently using the desired algorithms to approximate the sampling distribution of algorithms. ParamILS employs iterative local search (ILS) to exploit

¹Locality denotes the ability to generate neighboring solutions via a heuristically-informed function of one or more incumbent solutions [2].

desired algorithms; the intensification and capping strategies are developed to save algorithm evaluations. Offline hyperheuristics adopt high-level heuristics/metaheuristics, e.g., genetic programming [14], to generate low-level metaheuristic algorithms.

The above methods show two common points. First, they manipulate a fixed-length vectorized algorithm representation, i.e., a concatenation of categorical indexes of algorithmic components and their conditional hyperparameters; a specific algorithm template is required to determine the components' execution logic. This biases the design to a specific type of algorithms in the metaheuristic family. Second, given a target problem, most of them design from scratch. Although prior algorithms can be used as starting points, the knowledge regarding what motivates certain design decisions for a problem is untraceable. This knowledge may shed insights into future designs. These two points pose a clear gap toward fully discovering potentials across the metaheuristic family and fertilizing from prior design knowledge.

In this article, we bridge the gap by proposing the autoregressive learning-based designer (ALDes) for automated design of metaheuristic algorithms. We provide the following contributions in ALDes.

- 1) We formulate the metaheuristic algorithm design as a *sequence generation* task. To this we propose a new sequence representation for the algorithm. The representation tokenizes not only algorithmic components and hyperparameters but also “pointers.” Unlike the vector representation within a specific template, the pointers in our representation determine the execution logic of each component, and importantly, can form sequence, branch, and loop executions. This allows the sequence representation to express arbitrary algorithm structures without needing a specific template.
- 2) We harness an *autoregressive* generative network to handle the sequence generation task. Unlike current methods that orthogonally manipulate each entity of a fixed-length vector representation, we borrow the sequential generation ability from the transformer to autoregressively generate entities one at a time, conditioned on the previously generated ones. The autoregressive sequence generation pushes the boundary of automated algorithm design to a new level: designing algorithms with diverse lengths and structures, enabling to fully discover potentials over the metaheuristic family.
- 3) We learn from prior design experience by memorizing and retrieving prior knowledge within neurons of the proposed ALDes. Unlike current methods that handle each algorithm design task independently from scratch, we learn a factored embedding of a target problem and learn to generate algorithms upon the problem in an end-to-end fashion. Design knowledge accumulated in neurons of ALDes can be retrieved for algorithm design of new problems, by feeding the new problems. This paves the way to continual design of algorithms for open-ended problem solving.
- 4) We extensively examine ALDes on 23 pseudo-Boolean optimization (PBO) benchmarks and two real-world

applications. Results reveal that comparison baselines with a fixed algorithm structure are not eligible for all problems. ALDes, in contrast, discovers algorithms with different neighborhood definitions, search behaviors, and structures, reasonably fitting for different problems and outperforming all human-created baselines on 24 out of the 25 problems. Further experiments on continual algorithm design tasks illustrate ALDes's ability to leverage prior knowledge for future designs.

The remainder of this article contains as follows. Background and related work of automated design of metaheuristic algorithms in Section II, details of the proposed ALDes in Section III, experiments and applications in Section IV, and conclusion in Section V.

II. BACKGROUND AND RELATED WORK

A. Automated Design of Metaheuristic Algorithms

Without loss of generality, the offline automated design of metaheuristic algorithms aims to handle the following task [3]:

$$\begin{aligned} \arg \max_{A \in \mathcal{A}} \mathbb{E}_{\mathcal{I}}[\mathbb{E}_{\mathcal{G}}[g(A|i)]] \\ g \in \mathcal{G}, i \in \mathcal{I}_t, t = 1, 2, \dots, T \end{aligned} \quad (1)$$

where A refers to the algorithm to be designed, $\mathcal{A}$ is the design space constituting with algorithmic components and hyperparameters, and $g : \mathcal{A} \times \mathcal{I} \rightarrow \mathbb{R}$ is A 's performance obtained by a run of A on problem instance $i \in \mathcal{I}$. The performance can be solution quality, runtime, or anytime performance *etc.* [3].

Instance i is drawn from the target problem domain $\mathcal{I}_t$, where t refers to the t th algorithm design task. In particular, $T = 1$ indicates a *one-off* algorithm design task with a fixed target problem domain. The design needs to operate from scratch without prior experience to learn and use. $T > 1$ indicates *continual* algorithm design tasks with a series of (or open-ended) target problem domains. The design is allowed to accumulate knowledge from prior tasks, so as to fast adapt to new tasks. This case aligns with the pursuit of general and open-ended (lifelong) automated systems.

Since metaheuristic algorithms perform stochastic search, one needs to estimate the expected performance over $\mathcal{G}$; in practice this is done by multiple runs of A , resulting in multiple values of $g \in \mathcal{G}$. Since the distribution of i over $\mathcal{I}_t$ is unknown, i is usually drawn from a finite set of instances $\mathcal{I}_t^{\text{train}} \in \mathcal{I}_t$. Another finite set of instances $\mathcal{I}_t^{\text{test}} \in \mathcal{I}_t \setminus \mathcal{I}_t^{\text{train}}$ is used to test the designed algorithms.

For continuous problems, algorithmic components in the design space $\mathcal{A}$ can either be computational primitives, e.g., $+$, $-$, $*$, $/$, or existing operators, e.g., Gaussian mutation [15]. For discrete problems, there is not a clear distinction between primitives and operators, e.g., *swap* is a representative search operator and also a computational primitive. The design space with operators can be formed with good design choices from prior algorithms and users' domain expertise, thus has got strong track record in many work [16], [17], [18], [19], [20], [21]. In this article, we follow them and conduct automated design on the *operator* level.

Methods for offline automated design of metaheuristic algorithms mostly stem from the hyperparameter configuration field. They represent algorithmic components as categorical parameters; the categorical parameters and their conditional hyperparameters are concatenated to form a vector representation of the algorithm; an algorithm can be generated by manipulating the vector representation. Due to the black-box and NP-hard nature of the algorithm design task in (1), search-based methods are dominant in manipulating the vector representation of algorithms. For example, SMAC [8], [9] and MIP-EGO [22] use the Bayesian optimization to build a surrogate function that models the mapping from algorithms to their performance on a target problem. Irace [10] also employs model-based search. In particular, it proposes an iterative racing scheme. At each iteration, it first uses racing to select desired algorithms; then the desired algorithms are used to approximate the sampling distribution of algorithms; after that new algorithms are sampled from the distribution for the next iteration. Univariate norm distribution and discrete distribution are the default for numeral and categorical parameters in the vector representation, respectively, in these model-based methods [8], [10]. ParamILS [11] adopts the model-free ILS to exploit desired algorithms; the intensification and capping strategies are proposed to save the algorithm evaluations.

GP-based methods, mostly refer to tree-based GPs [14], conducted algorithm design on the computational primitive level, because of GP tree's expressiveness in representing programs over the primitives. They normally generated a single algorithmic operator instead of an entire algorithm (e.g., generating selection operators [23], [24]) [3], as representing an entire algorithm leads to a large tree and could be infeasible to find an appropriate tree over the large search space [25]. In contrast, this article focuses on generating entire algorithms on the operator level. There have been a few tree-based GP studies that generated algorithms at the operator level [26], [27]. However, they employed fixed evolutionary algorithm structures without investigating diversification among different metaheuristics [26], [27]. Cartesian GP [28] has also been employed to generate algorithms on the operator level but only supported generating linear and unfolded algorithms [29], [30], [31].

These design methods manipulate within specific algorithm templates. The template determines feasible execution logic of algorithmic components involved in the representation. For example, simulated annealing (SA) [32], particle swarm optimization (PSO) [33], and the genetic algorithm (GA) [34] templates were used to design new variants of SA [35], PSO [19], and GA [16], [18], [20], respectively. This limits to designing a certain type of metaheuristics rather than exploring the whole metaheuristic family. Furthermore, these search-based methods handle each algorithm design task independently from scratch. Although prior algorithms can be used as starting design points, the knowledge regarding what motivates certain design decisions for certain target problems is untraceable, losing insights and principles for future reuse. In this article, we dedicate to breaking through the two fundamental limitations in offline automated design

by proposing the autoregressive learning-based metaheuristic algorithm designer.

We note that reinforcement learning is another stream of design methods, but they are employed in online automated design in [20], [36], [37], [38], [39], and [40]. Using terminologies in reinforcement learning, the state often refers to the information from evaluating the performance of the designed algorithm in the current problem solving, e.g., features of the target problem instances and features of the algorithm's solutions to the instances; the action is adjusting certain components of the current algorithm; a policy is learned to generate an algorithm with a certain component composition to fit for the problem-solving state. Tabular reinforcement learning, as well as the modern deep Q-network [41] and proximal policy optimization [42], have been used to learn the policy [20], [36], [37], [40]. The policy results in an algorithm with adjustable components according to the search trajectory during problem solving. The structure of the designed algorithm is fixed.

B. Sequence Generation With Transformer

Transformer is a neural network architecture originally proposed for sequence-to-sequence learning in natural language processing [43]. Self-attention [44], [45] is the core module of transformer. It learns valuable features from the input, by capturing pairwise relations between tokens in the input sequence.² Specifically, it first linearly maps the input sequence $\mathbf{X} \in \mathbb{R}^{N \times d}$ (N tokens, each with a size of d) to a query, key, and value matrices $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$, respectively

$$\mathbf{Q} = \mathbf{X} \times \mathbf{W}^q, \mathbf{K} = \mathbf{X} \times \mathbf{W}^k, \mathbf{V} = \mathbf{X} \times \mathbf{W}^v \quad (2)$$

where $\mathbf{W}^q$, $\mathbf{W}^k$, and $\mathbf{W}^v \in \mathbb{R}^{d \times d'}$ are the learnable parameters that augment the features of the input sequence. Then, the pairwise relations are captured by

$$\mathbf{X}' = \text{softmax}\left(\frac{\mathbf{Q} \times \mathbf{K}^\top}{\sqrt{d'}}\right) \times \mathbf{V} \quad (3)$$

where the dot-product score function $\mathbf{Q} \times \mathbf{K}^\top$ scores the pairwise similarity between tokens; $\sqrt{d'}$ scales down the dot productions to stabilize the training; the alignment function $\text{softmax}(\cdot)$ normalizes the scores; and the pairwise relations are calculated by weighted sum of $\mathbf{V}$ with the normalized attention scores.

The self-attention can be conducted in S -head subspaces. That is, $\mathbf{X}$ is mapped with $\{\mathbf{W}_s^q, \mathbf{W}_s^k, \mathbf{W}_s^v\}_{s=1}^S$, respectively. Through (2) and (3), $\{\mathbf{X}'_s\}_{s=1}^S \in \mathbb{R}^{N \times (d'/S)}$ are obtained. These $\mathbf{X}'_s$ are concatenated to form $\mathbf{X}'$ via $\text{concat}(\mathbf{X}'_1, \mathbf{X}'_2, \dots, \mathbf{X}'_S) \times \mathbf{W}^0$, where $\mathbf{W}^0 \in \mathbb{R}^{d' \times d}$ are the learnable parameters. The residual connection and layer normalization are then applied to promote information flow

$$\mathbf{X}'' = \text{layer_norm}(\mathbf{X} + \mathbf{X}'). \quad (4)$$

²Tokens in the sequence should first go through an embedding process, if they are originally unstructured. Each token should be combined with a positional encoding, because the tokens in the sequence are ordered but the attention process is invariant to the order.

The position-wise feed-forward consisted of two linear transformations and a ReLU activation in between is then applied

$$\mathbf{X}''' = \max\left(0, \mathbf{X}'' \times \mathbf{W}_1^{\text{ffn}} + b_1\right) \times \mathbf{W}_2^{\text{ffn}} + b_2 \quad (5)$$

where $\mathbf{W}_1^{\text{ffn}}$ and $\mathbf{W}_2^{\text{ffn}}$ are the learnable parameters. b_1 and b_2 are the biases.

The vanilla transformer includes an encoder and a decoder. The encoder uses self-attention to learn valuable features from the input. The decoder uses self-attention to learn relations between tokens in the previously generated sequence segment, in turn to condition the autoregressive generation of the next token. Multiple encoder and decoder blocks can be stacked, respectively, to learn deep features. Refer to [43] for the complete transformer architecture.

As an alternative to recurrent neural networks [46], transformer offers strong sequential generation ability by using self-attention to learn through many shortcut relation paths between each pair of tokens in the sequence. This sequential generation ability is appealing for design tasks, e.g., designing neural architectures [47], [48], drug molecules [49], and radiofrequency waveforms [50], since the objects (sequences) to be designed are usually in various structures (lengths), and components of the object (tokens of the sequence) are dependent. The multitude of shortcut relation paths also benefit learning valuable features from complicated inputs (environment of the design tasks), which is significant to partial observable or black-box design environment [51]. In this article, we harness transformer's generation and learning ability to automated design of metaheuristic algorithms.

III. PROPOSED ALDES

Current automated design methodologies generate algorithms within a fixed structure, blocking potentially promising algorithms beyond that structure. Furthermore, they usually operate from scratch. Prior knowledge regarding what motivates certain design decisions for a problem is untraceable, losing insights and principles for future designs.

We propose the ALDes, to address these issues. Our idea is to formulate the metaheuristic algorithm design as a sequence generation task. By *autoregressive* learning to manipulate the *sequence* representation of algorithms, we are able to generate algorithms in various lengths and structures. Furthermore, we learn a factored embedding of a target problem and generate algorithms upon the problem in an end-to-end fashion. This allows prior design knowledge to be traceable for boosting future designs, by feeding the future problem.

We give the new formulation in Section III-A, then detail the workflow of ALDes in Section III-B, and describe how to learn from prior experience for open-ended problem solving in Section III-C.

A. Formulation

1) *Algorithm Design as Sequence Generation*: We formulate the automated design of metaheuristic algorithms as an autoregressive sequence generation task

$$\max_{\Theta} p_{\Theta}(A) = \sum_{n=1}^N \log p_{\Theta}(a_n | a_{1:(n-1)})$$

$$\text{s.t. } A = \arg \max_{A \in \mathcal{A}} \mathbb{E}_{\mathcal{I}}[\mathbb{E}_{\mathcal{G}}[g(A|i)]], \quad i \in \mathcal{I}, g \in \mathcal{G} \quad (6)$$

where Θ refers to the parameters of the autoregressive model; $A = (a_1, a_2, \dots, a_N)$ is the sequence representation of the algorithm to be designed; and $a_n, n = 1, 2, \dots, N$, is a token of the sequence; each token's probability p is conditioned on previous tokens; the joint probability is factorized as a product of the conditionals. Notations in the constraint are the same as those in (1).

We present two appealing features with the new formulation. First, we autoregressively generate entities (tokens) one at a time, conditioned on the previously generated ones. This is aligned with the nature that components of a metaheuristic algorithm are dependent and the components may have conditional hyperparameters.³ Second, we can generate sequences in various lengths, enabling us to generate diversified algorithms. In comparison, the previous design often orthogonally manipulates each entity of a fixed-length algorithm, falling to capture the dependence and provide diversification.

2) *Representing Algorithm as Sequence*: We propose a new sequence representation of metaheuristic algorithms to fit for the formulation. The sequence representation tokenizes not only algorithmic components and hyperparameters but also *pointers* and *conditions*. Specifically, we treat each component, hyperparameter, pointer, and condition as a token of the sequence, respectively. A component token is followed by its hyperparameter (if any) tokens, a pointer token, and a condition token. They form a snippet. A number of snippets are arranged to form the sequence

$$\begin{aligned} A &= (S_1, S_2, \dots) \\ \text{with } S_l &= (a^{\text{comp}}, a^{\text{param}_1}, \dots, a^{\text{param}_C}, a^{\text{ptr}}, a^{\text{cond}}) \\ l &= 1, 2, \dots \end{aligned} \quad (7)$$

where A is the sequence, S_l is the l th snippet, each a is a token, and C is the number of hyperparameters of component a^{comp} .

In a snippet, pointer a^{ptr} determines the execution flow after running component a^{comp} . We define three pointers, forward, iterate, and fork. The first makes the execution flow head to the component of the next snippet; the middle makes a^{comp} iterate; the final forks the flow head to the next component and certain subsequent component(s). The *fork* will bring a control parameter token following it, to determine which subsequent components to link to. Condition a^{cond} regulates the computational cost of a^{comp} . We define two types of conditions, i.e., count-conditions (the number of execution times) and event-conditions (executing until an event happened).⁴ Our new representation can form sequence, branch, and loop executions, thereby expressing various algorithm structures without a template. This cannot

³For example, a neighborhood search component has hyperparameters determining the neighborhood region, the uniform mutation component has a hyperparameter determining the mutation probability.

⁴Event-conditions normally require problem-specific knowledge. For example, for the event of reaching a local optima, the definition of the local optimal (e.g., a solution quality threshold) should be predefined. We do not consider event-conditions in the experiment of this article due to the lack of prior knowledge of the target problems.

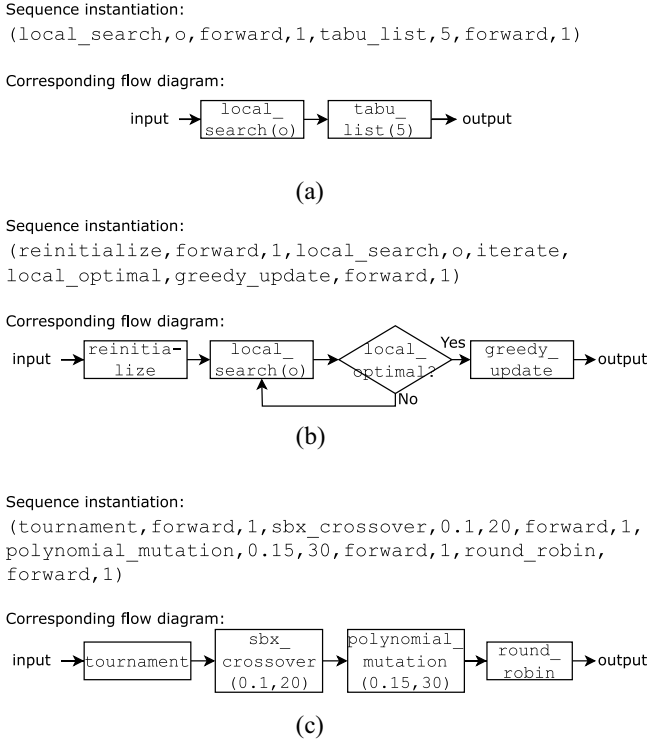


Fig. 1. Three instantiations of the sequence representation. In each iteration, (a) TS executes two components, `local_search` and `tabu_list`. The `local_search` (with hyperparameter `o` defines the neighborhood region) executes once (count condition 1), then the flow forward to update the `tabu_list` (hyperparameter 5 refers the list's length) once (count condition 1), after that the flow forward to the next algorithm iteration. Likewise, in (b), the `local_search` iterates until reaching a local optimal (event condition `local_optimal`). In (c), 0.1, 20 (0.15, 30) are the crossover (mutation) probability and distribution, respectively.

be done by the common practice of fixed-length vector representation. Fig. 1 shows instantiations of our sequence representation.

B. Workflow

We harness an autoregressive generative network to handle the sequence generation task of (6). Fig. 2 shows the workflow, detailed next.

1) *Architecture*: From Fig. 2, the network contains four main modules, i.e., problem embedding, sequence embedding, transformer, and masked sampling. We first introduce these modules, respectively, then illustrate the workflow.

Problem embedding embeds the unstructured (e.g., symbolic expression) target problem into ALDes. We elaborate this module in Section III-C, as it is only available for learning from prior experience in *continual* algorithm design tasks (i.e., $T > 1$ in (1)) and is unnecessary for *one-off* tasks. In continual tasks, ALDes must recognize the target problems in different tasks by embeddings. Thus, it can learn from the design experience on prior problems and extract relevant experience for the design on new problems. While in one-off algorithm design tasks, it does not, as there is only a single fixed target problem domain.

Sequence embedding embeds the input sequence (A in (7)) into ALDes. First, we convert the sequence's tokens to the one-hot format, each is of dimension M , where M is the number of all candidate tokens, including algorithmic components, hyperparameters, pointers, conditions, and an “end” token that indicates the termination of sequence generation. To enable one-hot tokens, continuous hyperparameters are discretized; while components, pointers, and conditions are naturally discrete. Then, following the common practice of sequence transduction, we learn a linear transformation $\mathbf{W}^{\text{seq}} \in \mathbb{R}^{M \times d}$ to embed the one-hot tokens:

$$A'' = A' \times \mathbf{W}^{\text{seq}} \quad (8)$$

where $A' \in \mathbb{R}^{N \times M}$ is the one-hot format of the input sequence A and N is the number of tokens in the current sequence. Afterward, the tokens go through sinusoidal positional encoding to enable position information [43]

$$\begin{aligned} A'''_{(n,\mu)} &= A''_{(n,\mu)} + \sin\left(\frac{n}{10000^{\frac{\mu}{d}}}\right) \\ A'''_{(n,\sigma)} &= A''_{(n,\sigma)} + \cos\left(\frac{n}{10000^{\frac{\sigma}{d}}}\right) \end{aligned} \quad (9)$$

where $A''_{(n,\mu)}$ and $A''_{(n,\sigma)}$ are the μ th and σ th dimensions of the n th token in sequence A'' , respectively. μ and σ refer specifically to even and odd dimensions, respectively.

The transformer module learns features and captures dependencies over tokens of the current sequence, predicting the next token. We use the transformer due to its well-established track records in sequence generation, by self-attention to learn from many shortcut relation paths between tokens of the sequence. The Multihead Attention, Add and Norm, and Feed-Forward are calculated by (2)–(5), respectively. A'' acts as the input of (2). Linear is a learnable linear transformation $\mathbf{W}^l \in \mathbb{R}^{d \times M}$ applied to the final token of its input

$$\mathbf{y} = A''_{(N)} \times \mathbf{W}^l \quad (10)$$

where $A''_{(N)} \in \mathbb{R}^d$ is the final token of the input A'' of linear and $\mathbf{y} \in \mathbb{R}^M$ is the output of the transformer module.

Masked sampling predicts the next token by sampling over candidate ones. Usual sequence generation tasks involve homogeneous tokens (e.g., words in natural language sequences); each of the tokens should be considered in sampling. Our task, in contrast, involves heterogeneous tokens with specific orders in the sequence (7). Given this, we can mask unexpected types of tokens in sampling: all tokens but components are masked at the first round of token prediction. This ensures the sequence starts from a component. Only hyperparameter tokens are unmasked at subsequent rounds, if the component has hyperparameters. Only pointer tokens are unmasked, if the component is hyperparameter-free or the hyperparameters have been generated. Likewise, only condition tokens are unmasked after a pointer token. Only component tokens and the end token are unmasked after a condition token. These principles determine an M -dimensional

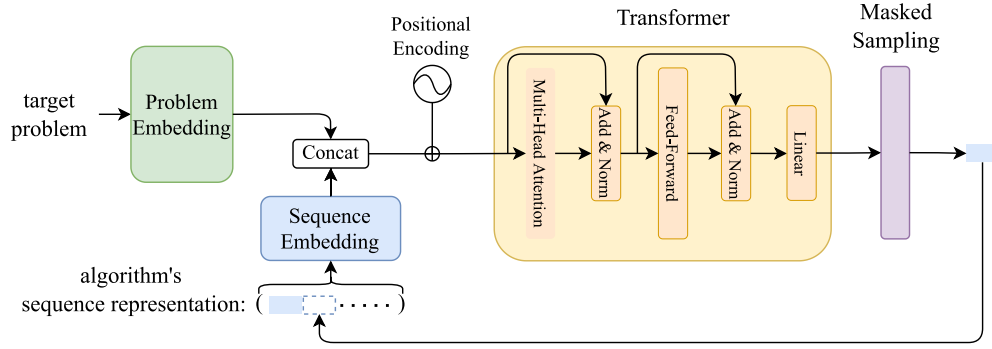


Fig. 2. Workflow of the proposed ALDes.

vector mask $\mathbf{z}$ with 1 for unmasked tokens and $-\infty$ for masked tokens. The sampling follows a discrete distribution given by:

$$\text{softmax}(\mathbf{y} \circ \mathbf{z}) \quad (11)$$

where $\circ$ is the Hadamard product. The masked sampling greatly narrows the sampling space, thus easing the sequence generation task.

Given the above modules, the workflow starts from inputting the algorithm's sequence representation to sequence embedding. The initial sequence is made up of a "begin" token, which prompts ALDes to begin the token prediction. After embedding, the sequence goes through positional encoding, transformer, and masked sampling, outputting the next token. The outputted token is then put in the sequence, starting the next round of token prediction. The workflow terminates once the *end* token is outputted.

2) *Training*: Given (6), we train ALDes to maximize the probability of generating the algorithm with the maximum expected performance on the target problem

$$J(\Theta) = \mathbb{E}_{p_{\Theta}(a_{1:N})}[R] \quad (12)$$

where Θ refers to learnable parameters, including weights of the linear transformation in sequence embedding ($\mathbf{W}^{\text{seq}}$ in (8)) and parameters in the transformer [$\mathbf{W}^q$, $\mathbf{W}^k$, and $\mathbf{W}^v$ in (2), $\mathbf{W}_1^{\text{ffn}}$ and $\mathbf{W}_2^{\text{ffn}}$ in (5), and $\mathbf{W}^l$ in (10)]. $R = \mathbb{E}_{\mathcal{I}}[\mathbb{E}_{\mathcal{G}}[g(A|i)]]$, $i \in \mathcal{I}$, $g \in \mathcal{G}$, and $A = (a_{1:N})$, is the expected performance, with the same notations as those in (1).

Since R is nondifferentiable, we resort to a policy gradient method to iteratively update Θ . Specifically, we use the proximal policy optimization [42]

$$J(\Theta) = \mathbb{E}_k \left[\min \left(h_k(\Theta) \hat{R}_k, \text{clip}(h_k(\Theta), 1 - \epsilon, 1 + \epsilon) \hat{R}_k \right) \right] \\ \text{with } h_k(\Theta) = \frac{\prod_{n=1}^N p_{\Theta}(a_n^k | a_{1:(n-1)}^k)}{\prod_{n=1}^N p_{\Theta_{\text{old}}}(a_n^k | a_{1:(n-1)}^k)} \quad (13)$$

where k is the index of the k th sequence (i.e., algorithm) in a batch; $h_k(\Theta)$ is the weight of importance sampling [52]; Θ_{old} refers to the parameters before update; a_n^k is the n th token of algorithm k ; $\hat{R}_k = R_k - b$, in which R_k is the k th algorithm's expected performance, b is a weighted moving average baseline that reduces the variance of gradient estimate; and ϵ is the clip threshold. The proximal policy optimization allows

multiple epochs of minibatch updates, greatly enhancing the training effectiveness concerning standard policy gradient.

C. Learning From Prior Experience

Unlike search-based design methods that handle each algorithm design task independently from scratch, ALDes can learn and accumulate design knowledge over time in *continual* scenarios to fast adapt to new problems. This aligns with the pursuit of general and open-ended automated systems. To this, we 1) embed the target problems to let ALDes recognize them, such that ALDes can retrieve relevant prior knowledge for new problems and 2) regularize ALDes's training to accumulate prior knowledge while adapting to new problems.

For point 1), we use the problem embedding module (Fig. 2). Intuitively, the embedding might be done by representation learning, e.g., word2vec [53] to embed the textual description of the target problem, mathematical language processing [54] to embed the symbolic expression of the target problem. But these methods are incomplete, because hard problems are nontrivial to be described by text, symbolic expression is multimodal,⁵ and many problems do not have symbolic expressions (e.g., black-box ones).

Given this, we choose a factored embedding, i.e., converting the target problem to a factor representation. The representation is a vector of factors describing the problem's landscape features. The factors are calculated by exploratory landscape analysis [55] in two steps: 1) Sampling the problem's solution space and evaluating the sampled solutions' fitness. The Sobol [56] and random walk samplings are used for continuous and discrete problems, respectively, following [57] and [58]. 2) Calculating landscape features with the samples. The features are chosen following [57] (detailed in the experiment). Note that, the sampling does *not* incur additional computational overhead, as we randomly select solutions from the sampled ones and reuse them as initial solutions of the generated algorithm A , when estimating A 's expected performance R (12). Given a continual algorithm design task, the factored embeddings of all target problems are calculated by the same features, in order to build an identical embedding space to recognize problems for similarity-based inference.

⁵A problem can have different symbolic formulations.

The factor representation goes through a learnable linear transformation $\mathbf{W}^{\text{probl}} \in \mathbb{R}^{U \times d}$, where U is the number of factors. Afterward, it serves as a token and concatenates with tokens of the embedded sequence (Fig. 2). The subsequent transformer attends the factored representation of the target problem and biases the algorithm design upon the problem. The factored embedding does not rely on any specific form of the input target problem, e.g., textual or symbolic expression, thus is a general problem embedding for algorithm design.

For point 2), we resort to the elastic weight consolidation (EWC) [59]. It can estimate the importance of ALDes's each parameter to accumulating prior design knowledge, and slow down updating the important parameters to adapt to new tasks, so as to balance the accumulation and adaption. Accordingly, the training of ALDes is regularized by [59]

$$J'(\Theta) = J_t(\Theta) + \sum_{r=1}^R \frac{\lambda}{2} F_r \|\Theta_r - \Theta_{r,t-1}^*\|^2 \quad (14)$$

where Θ refers to learnable parameters, including these from (12) plus $\mathbf{W}^{\text{probl}}$; $J_t(\Theta)$ is the loss of the current algorithm design task and is calculated by (13); F_r is the Fisher information matrix estimating the importance of the r th parameter to memorizing prior knowledge; $\Theta_{r,t-1}^*$ is the r th (learned) parameter for previous tasks; and λ sets the weight. We resort to this well-established regularization method, because it is scalable and will not incur overload in terms of memory and model scale, concerning replay and parameter isolation methods [60].

IV. EXPERIMENTS AND APPLICATIONS

We do experiments and applications to evaluate 1) the proposed ALDes's efficiency in generating high-performance and diversified metaheuristic algorithms, and the necessity of such diversification to fit for different problem solving and 2) the efficiency of ALDes in learning from prior design experience to facilitate future designs.

A. Setup

1) *Target Problems*: They include the representative PBO numerical benchmarks [61] and two real-world problems. PBO is a well-established discrete optimization problem suite with various fitness landscapes. Twenty-three PBO functions are used, including OneMax, its harmonic weighted version, W-model extensions (F1 and F3–F10), LeadingOnes and its W-model extensions (F2 and F11–F17), low autocorrelation binary sequences (F18), Ising models (F19–F21), maximum independent vertex set (F22), and N-queens (F23). Each function is treated as an independent target problem with four instances in different solution space dimensions: 100, 225, 400, and 625d.

The real-world problems are for investigating ALDes's practicality. The first is the discrete, nonseparable beamforming problem from a reconfigurable intelligent surface (RIS)-aided communication system. It involves ten instances that differ in the number of RIS elements. The second is the power system restoration problem with a highly constrained binary

solution space. It contains two instances that differ in the locations of black start (BS) and nonblack start (NBS) units. Metaheuristics could handle the discrete, highly constrained, and nonseparable challenges. The automated design could customize efficient algorithms from various options.

2) *Baselines*: Baselines span various algorithms across the metaheuristic family for a comprehensive comparison. They include ILS [62], SA [32], Tabu search (TS) [63], and GA [34]. The neighborhood structure of ILS, SA, and TS is a common practice of 1d distinct from the current solution. ILS uses random reinitialization as global perturbation and “no improvement in three consecutive iterations” as acceptance criterion. GA employs uniform crossover and standard bit mutation unless stated otherwise. of these baselines, including SA's initial temperature and cooling schedule, TS's Tabu list length and Tabu tenure, and GA's crossover probability η_c and the number of bits to be flipped η_m are configured by irace [10] for each target problem, according to settings of [22]. The analytical baseline CPLEX is used for real-world problems to assess ALDes's practicality.

3) *Implementation Details*: We implement the proposed ALDes as follows.

- 1) *Size*: The output dimension is 32 (i.e., $d = d' = 32$) for all learnable parameters except for $\mathbf{W}^l$. Eight heads are used in the attention. This model structure is straightforward. More complex structures may boost performance but are not the focus of this article.
- 2) *Training*: For each PBO problem, the 100, 225, and 400d instances are for training; for each real-world problem, half of its instances are for training. On each problem, ALDes is trained for 100 epochs with a batch size of 16 algorithms generated per epoch. Update iteration=5 within each proximal policy optimization. For evaluating algorithms' expected performance (R in (12)), each generated algorithm runs five times per training instance with 5000 function evaluations (FEs) per run. The performance is measured by the best solution fitness, and expected performance is averaged over all runs. More evaluation budget could stabilize training but increase computation load. We investigate the impact of the evaluation budget on the training in Section S.II-A of the Supplementary Material. An algorithm contains up to 6 components to avoid unnecessary complexity. Adam optimizer [64] with annealing learning rate starting from $5e - 5$, $\epsilon = 0.2$, and $\lambda = 200$ are used,⁶ following the common practice of [42], [59], and [64], respectively. The design space is detailed in Section S.I-A of the Supplementary Material.
- 3) *Testing*: The 625d instance of each PBO problem and the remaining instances of each real-world problem are used for testing. The algorithm inferred from the trained

⁶Specifically, the initial learning rate of the Adam optimizer determines the convergence speed at the initial stage of training. A large rate results in a high convergence speed at the initial stage, and vice versa. The clip threshold ϵ of the proximal policy optimization determines the range of clip during the model (ALDes) update. A larger ϵ value allows updating by a larger margin, and vice versa. The weight λ of the regularization in EWC determines the importance of prior knowledge to the model update. A larger weight indicates a higher importance, and vice versa.

TABLE I

PERFORMANCE (MEAN $\pm$ STD) OVER ALL RUNS ON THE TEST INSTANCE OF EACH PBO PROBLEM. Alg*'s PERFORMANCE IS AVERAGED OVER FIVE ALGORITHMS INFERRED FROM ALDES'S FIVE TRAINING TRIALS WITH DIFFERENT SEEDS. BEST RESULTS ACCORDING TO PAIR-WISE WILCOXON SIGN TEST (5% SIGNIFICANCE LEVEL) ARE WITH GRAY BACKGROUND

	ILS	SA	TS	GA	Alg*
F1	609.40 $\pm$ 2.87E+00	388.30 $\pm$ 2.12E+01	398.10 $\pm$ 1.46E+02	452.63 $\pm$ 7.96E+00	625.00 $\pm$ 0.00E+00
F2	93.67 $\pm$ 8.97E+01	14.77 $\pm$ 1.04E+01	166.17 $\pm$ 1.39E+02	54.40 $\pm$ 3.14E+00	158.67 $\pm$ 2.29E+01
F3	192550.47 $\pm$ 1.13E+05	126377.90 $\pm$ 2.71E+06	141253.13 $\pm$ 1.58E+07	146910.90 $\pm$ 1.27E+06	195625.00 $\pm$ 0.00E+00
F4	312.00 $\pm$ 0.00E+00	215.63 $\pm$ 1.30E+01	241.83 $\pm$ 6.89E+01	250.83 $\pm$ 3.80E+00	312.00 $\pm$ 0.00E+00
F5	555.37 $\pm$ 1.96E+00	354.90 $\pm$ 1.35E+01	362.87 $\pm$ 1.37E+02	413.83 $\pm$ 1.17E+01	562.00 $\pm$ 0.00E+00
F6	203.80 $\pm$ 5.34E+00	155.50 $\pm$ 4.25E+00	183.17 $\pm$ 2.31E+01	170.33 $\pm$ 2.30E+00	207.90 $\pm$ 9.31E-02
F7	504.17 $\pm$ 3.35E+01	394.57 $\pm$ 1.58E+01	495.17 $\pm$ 4.61E+01	386.30 $\pm$ 2.01E+01	510.57 $\pm$ 2.29E+01
F8	303.40 $\pm$ 1.28E+00	195.67 $\pm$ 5.09E+00	157.83 $\pm$ 6.02E+01	227.47 $\pm$ 1.77E+00	314.00 $\pm$ 0.00E+00
F9	591.40 $\pm$ 1.26E+02	385.80 $\pm$ 2.38E+01	311.47 $\pm$ 1.50E+02	452.23 $\pm$ 8.81E+00	625.00 $\pm$ 0.00E+00
F10	367.00 $\pm$ 3.72E+01	378.17 $\pm$ 1.85E+01	315.67 $\pm$ 1.89E+02	450.87 $\pm$ 1.06E+01	598.50 $\pm$ 5.58E+01
F11	92.67 $\pm$ 3.05E+02	14.67 $\pm$ 9.62E+00	170.73 $\pm$ 1.25E+02	54.20 $\pm$ 6.58E+00	161.87 $\pm$ 5.10E+01
F12	93.90 $\pm$ 1.20E+02	14.43 $\pm$ 1.24E+01	166.43 $\pm$ 1.50E+02	55.10 $\pm$ 6.30E+00	164.20 $\pm$ 3.95E+01
F13	23.57 $\pm$ 9.61E+01	17.93 $\pm$ 7.33E+00	6.63 $\pm$ 3.00E+01	44.07 $\pm$ 3.58E+00	89.63 $\pm$ 9.07E+00
F14	17.30 $\pm$ 2.04E+01	11.50 $\pm$ 7.98E+00	4.27 $\pm$ 1.35E+01	44.43 $\pm$ 4.74E+01	47.57 $\pm$ 4.19E+01
F15	12.17 $\pm$ 1.11E+01	8.00 $\pm$ 1.87E+00	5.30 $\pm$ 1.58E+01	28.30 $\pm$ 2.29E+00	35.20 $\pm$ 1.08E+01
F16	18.27 $\pm$ 3.99E+01	13.70 $\pm$ 1.53E+01	7.73 $\pm$ 4.26E+01	52.73 $\pm$ 6.55E+00	67.83 $\pm$ 1.08E+01
F17	16.00 $\pm$ 7.93E+00	12.17 $\pm$ 1.25E+01	5.83 $\pm$ 1.08E+01	42.27 $\pm$ 4.30E+01	44.27 $\pm$ 1.44E+02
F18	3.73 $\pm$ 1.30E-02	2.13 $\pm$ 3.26E-04	3.76 $\pm$ 2.00E-02	1.51 $\pm$ 1.32E-03	4.28 $\pm$ 1.43E-02
F19	542.60 $\pm$ 6.96E+01	389.87 $\pm$ 3.09E+01	477.60 $\pm$ 2.22E+02	401.13 $\pm$ 8.81E+00	593.00 $\pm$ 2.43E+01
F20	1052.33 $\pm$ 1.64E+02	723.53 $\pm$ 4.79E+01	865.27 $\pm$ 2.87E+02	762.00 $\pm$ 5.85E+01	1110.47 $\pm$ 5.18E+02
F21	1581.93 $\pm$ 5.57E+02	1068.20 $\pm$ 1.29E+02	1303.73 $\pm$ 4.57E+02	1129.07 $\pm$ 6.17E+01	1724.13 $\pm$ 4.42E+02
F22	247.50 $\pm$ 1.78E+01	-16541.93 $\pm$ 1.65E+07	-34324.87 $\pm$ 1.29E+08	-55636.10 $\pm$ 4.60E+06	273.67 $\pm$ 2.40E+01
F23	-216.90 $\pm$ 4.33E+03	-17518.27 $\pm$ 4.30E+05	-18829.20 $\pm$ 8.02E+05	-13670.17 $\pm$ 6.85E+04	23.47 $\pm$ 2.57E-01

ALDes is compared with the baselines on test instances. All algorithms perform population-based search (population size = 50) for fair peer comparison. Each algorithm runs 30 times per problem with 50 000 FEs per run.

B. Efficiency in Generating High-Performance and Diversified Algorithms

In this section, we train the proposed ALDes from scratch on each PBO problem, *respectively*, to simulate one-off algorithm design tasks. Table I presents results on *test* problem instances, where Alg* is the algorithm inferred from the trained ALDes; larger values refer to better performance.

F1 and F3–F10 are OneMax, its Harmonic weighted version, and W-model extensions, respectively. OneMax can be solved by greedily flipping each bit from left to right. But the baselines, without considering such ordinal attribute in their neighborhood structure, fail to reach optima in the 625*d* solution space. The algorithm Alg* generated by ALDes (see Algorithm 1,⁷ with its sequence representation and convergence curve given in Figs. S2(a) and S3(a) of the Supplementary Material, respectively) is a variable neighborhood search (VNS) [65]-style algorithm utilizing two neighborhood structures: one is 1*d* distinct from the current solution (`reset_n`, $n = 1$); the other is determined by the uniform crossover of probability 0.7 and random reset of probability 0.1. Furthermore, the neighborhood structures are exploited by population-based search (roulette wheel for choosing solutions to search from, and pairwise selection for updating solutions). Such variable neighborhood structures and

Algorithm 1: Pseudocode of the Alg* for F1

Input: initial solutions S

```

1 while algorithm not terminate do
2   while count_5%*FE not met do
3      $S = \text{roulette\_wheel}(S)$ 
4      $S_{\text{new}} = \text{reset\_n}(S, n = 1)$ 
5      $S = \text{pairwise\_select}(S, S_{\text{new}})$ 
6   while count_5%*FE not met do
7      $S = \text{roulette\_wheel}(S)$ 
8      $S_{\text{new}} = \text{cross\_uniform}(S, p = 0.7)$ 
9      $S_{\text{new}} = \text{reset\_rand}(S_{\text{new}}, p = 0.1)$ 
10     $S = \text{pairwise\_select}(S, S_{\text{new}})$ 

```

Output: best solution from S

diversity maintenance ability enable Alg* reaching optima in all 30 runs within the limited FEs. Fig. 3(a) shows the training curve. ALDes converges well on this simple problem. Similar results are observed on F3 [Table I, Fig. 3(b)], the linear weighted version of F1. On F4–F10, the W-model extensions of F1 with dummy bits, neutrality, epistasis, and fitness perturbation in the landscape, ALDes shows constant outperformance.

F2 and F11–F17 are LeadingOnes and its W-model extensions. They are more difficult than the OneMax series due to their nonseparability, thus the baselines' results dramatically degrade. An exception is TS, which performs well on F2 and dummy bit variants (F11 and F12) but deteriorates on F13–F17. This implies that locating local optima by the Tabu list is insufficient to handle neutrality, epistasis, and fitness perturbation. ALDes overall works better. Specially, we observe interesting results on F13. The algorithm Alg* generated by ALDes (Algorithms 2, with its sequence representation, convergence curve, and performance boxplot given in Figs. S2(b),

⁷ALDes is trained five trials with different seeds. This results in five algorithms by inference from each trial (averaged performance is given in Table I). We detail the algorithm with the best performance to illustrate ALDes's potential. The same goes for Algorithms 2–6.

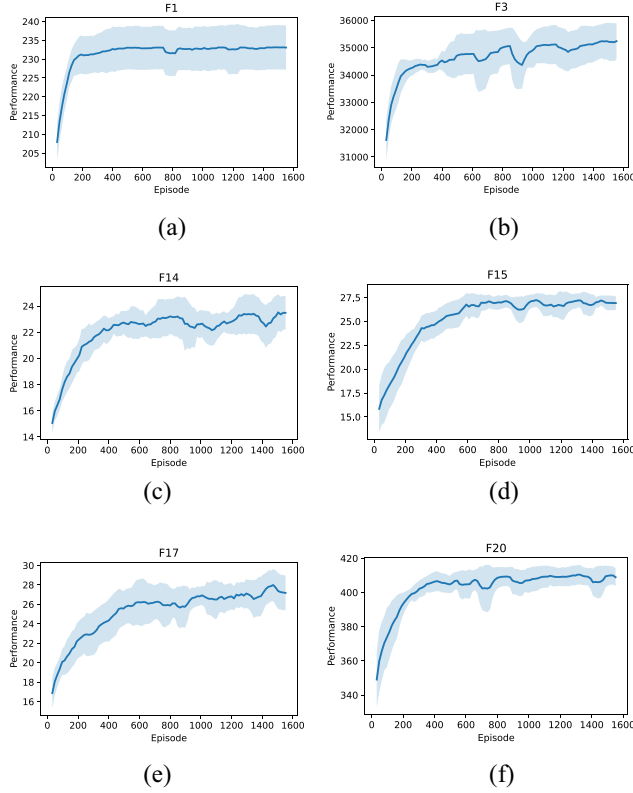


Fig. 3. ALDes's training performance versus episodes. Averaged over five trails. (a) F1. (b) F3. (c) F14. (d) F15. (e) F17. (f) F20.

S3(b), and S4 of the Supplementary Material, respectively) is a GA variant that always selects offspring S_{new} for the next generation (always_select). This unbiased algorithm makes sense to F13 with neutral landscape. The GA baseline is configured with $\eta_c = 0.05$ and $\eta_m = 0.05$ for F13. This relative steady-state search is also reasonable for neutrality, thus yields the second-best result. On F15, ALDes generates an ILS-style algorithm (Algorithms 3, with its sequence representation, convergence curve, and performance boxplot given in Figs. S2(c), S3(c), and S4 of the Supplementary Material, respectively), in which the local search of two-point crossover and random reset with probability of 0.1 iterates $10\% \times \text{FES}$ times, follows by global perturbation (reinitialize). This ILS variant much outperforms the ILS baseline. The larger magnitude of local search (by crossover and mutation) contributes to this, for the rugged and deceptive landscape. This observation is similar to that in [61]. For training efficiency, results on F14, F15, and F17, the three hardest problems in the LeadingOnes category, are shown in Fig. 3(c)–(e), respectively. Despite fluctuation, ALDes converges on training and performs well on test (Table I), which confirms the efficiency.

F19–F21 are Ising Models. They are NP-hard quadratic Hamiltonian energy functions of d spins up or down (d binary bits) in a lattice [66]. ALDes performs the best on these problems. For illustration, the generated algorithm Alg* for F20 is given in Algorithm 4; its sequence representation, convergence curve, and performance boxplot are shown in Figs. S2(d), S3(d), and S4 of the Supplementary Material, respectively. It

Algorithm 2: Pseudocode of the Alg* for F13

Input: initial solutions S
1 while algorithm not terminate **do**
 2 $S = \text{roulette_wheel}(S)$
 3 $S_{new} = \text{cross_n}(S, n = 2)$
 4 $S_{new} = \text{reset_rand}(S_{new}, p = 0.1)$
 5 $S = \text{always_select}(S, S_{new})$

Output: best solution from S

Algorithm 3: Pseudocode of the Alg* for F15

Input: initial solutions S
1 while algorithm not terminate **do**
 2 **while** $\text{count_}10\% \times \text{FE}$ not met **do**
 3 $S = \text{tournament}(S)$
 4 $S_{new} = \text{cross_n}(S, n = 2)$
 5 $S_{new} = \text{reset_n}(S_{new}, n = 1)$
 6 $S = \text{piarwise_select}(S, S_{new})$
 7 $S = \text{reinitialize}(S)$

Output: best solution from S

Algorithm 4: Pseudocode of the Alg* for F20

Input: initial solutions S
1 while algorithm not terminate **do**
 2 **while** $\text{count_}10\% \times \text{FE}$ not met **do**
 3 $S = \text{roulette_wheel}(S)$
 4 $S_{new} = \text{reset_n}(S, n = 1)$
 5 $S = \text{round_robin_select}(S, S_{new})$
 6 **while** $\text{count_}10\% \times \text{FE}$ not met **do**
 7 $S = \text{roulette_wheel}(S)$
 8 $S_{new} = \text{cross_uniform}(S, p = 0.9)$
 9 $S_{new} = \text{reset_rand}(S_{new}, p = 0.1)$
 10 $S = \text{round_robin_select}(S, S_{new})$

Output: best solution from S

is a VNS-style algorithm with two neighborhood structures. One is a small neighborhood of one bit distinct from the current solution; the other is a large neighborhood determined by the uniform crossover (probability of 0.9) and random reset (probability of 0.1). The neighborhood structures are exploited by population-based search (roulette wheel for choosing solutions to search from, and round robin for updating solutions). In comparison, the competitors with a single search structure are inferior. This suggests that the alternation of small and large neighborhood structures, together with population-based search, benefit the search of highly-coupled spins in Ising models. This observation is similar to that in [67], in which a slow quantum adiabatic, scheduled by the global Monte Carlo tree search, got remarkable performance. Eligible training performance is observed in Fig. 3(f).

ALDes also achieves leading performance on F18, F22, and F23. On F18, ALDes reaches 4.28 within 50 000 FEs, which is even better than the target value (4.26) obtained by 12 representative metaheuristic algorithms, each within $5 \times d^2$ ($d = 625$) FEs, in [61]. On F22, SA, TS, and GA's results are ineligible, implying that they may fall into the strong basin of attraction of this problem. Similar results are observed on F23, in which ALDes reaches 23.47, very close to the target value (24) in [61].

Put together, we observe three significant findings. First, the comparison on 625*d* test instances, which is of much higher dimension than ALDes's training instances (100, 225, and 400*d*), has placed ALDes at a disadvantage. Even so, ALDes wins the best on 22 out of 23 problems, and in most problems, outperforms competitors by a large margin. This clearly confirms ALDes's efficiency in generating high-performance algorithms. Second, the baselines' performance ranks are relatively balanced across the problems. This supports our motivation that a single fixed algorithm structure is not always the best for different problem solving. Third, ALDes generates diversified algorithms for different problems, e.g., the VNS-, ILS-, and GA-style ones with distinct behaviors (Algorithms 1–4). It also generates algorithms beyond intuition, such as the unbiased one for handling neutrality (Algorithm 2). The generated algorithms perform reasonably well on corresponding problems, as analyzed above. All these clearly demonstrate ALDes's efficiency in generating diversified algorithms across the metaheuristic family to suit different problem solving.

C. Efficiency in Learning From Prior Experience

In this section, we *continually* train the proposed ALDes on a series of PBO problems to simulate continual algorithm design tasks. We construct two continual tasks from the PBO suite: Task 1, comprising six OneMax versions (F1–F6) and one LeadingOnes (F7), is relatively easy. Task 2, consisting of F1, F2, F11, F18, F19, F22, and F23 from different categories (e.g., OneMax, LeadingOnes, Ising, etc), is challenging, which augments concept drift⁸ across the problems. For each task, starting from scratch, ALDes is trained on the problems one by one using (14) and the same settings as outlined in Section IV-A3. Factors for problem embedding are detailed in Section S.I-B of the Supplementary Material.

To examine ALDes's ability to accumulate prior knowledge, we compare it to the variant without EWC [i.e., train using (13), other settings keep constant]. This variant behaves akin to *fine tuning*, i.e., adapting to new tasks without considering prior ones. Fig. 4 reports the results. In task 1, with the training progress along the problems, ALDes with EWC constantly maintains its performance on previous problems, while the variant without EWC fails to do so on F1 and F2. This trend is more pronounced in the harder task 2, where the performance without EWC drops by half on F2, F11, and F18 as training progresses. ALDes with EWC performs relatively stable, showing its capability for knowledge accumulation.

We further investigate whether the accumulated knowledge can facilitate the design on future problems. Fig. 5 depicts training curves of F11, F18, F19, and F22 in task 2. Continual training on F11, F18, and F19 starts from much higher performance than training from scratch. The accumulated knowledge contributes to this fast adaption. Performance improves further as training progresses (see insets in Fig. 5). The adaption is not observed on F22, indicating significant concept drift between its landscape (with a strong basin

⁸The task's problems come from heterogeneous categories, inducing serious distribution drift across the problems.

		F1	F2	F3	F4	F5	F6	F7
After Training	F1	625.00						
	F2	625.00	110.13					
	F3	588.87	148.07	186551.00				
	F4	625.00	139.70	195625.00	312.00			
	F5	625.00	143.97	195625.00	312.00	562.00		
	F6	625.00	137.77	195625.00	312.00	562.00	206.07	
	F7	625.00	137.33	195625.00	312.00	562.00	206.67	502.37

(a)

		F1	F2	F3	F4	F5	F6	F7
After Training	F1	625.00						
	F2	588.30	114.30					
	F3	621.77	118.93	195621.37				
	F4	622.10	118.73	195620.37	312.00			
	F5	621.87	114.66	195623.00	312.00	562.00		
	F6	625.00	112.33	195621.63	312.00	562.00	206.80	
	F7	432.03	115.51	195621.67	312.00	562.00	206.40	509.50

(b)

		F1	F2	F11	F18	F19	F22	F23
After Training	F1	625.00						
	F2	624.56	151.86					
	F11	624.46	157.70	164.46				
	F18	624.60	159.20	162.40	4.46			
	F19	624.66	157.13	165.36	4.48	592.86		
	F22	625.00	162.16	164.03	4.07	590.73	269.13	
	F23	625.00	161.40	107.23	4.03	548.00	271.80	22.43

(c)

		F1	F2	F11	F18	F19	F22	F23
After Training	F1	625.00						
	F2	556.40	163.90					
	F11	472.20	109.90	107.76				
	F18	624.96	77.33	69.46	4.32			
	F19	622.96	156.83	78.13	4.06	591.40		
	F22	625.00	26.76	110.80	1.84	537.60	271.80	
	F23	622.83	27.40	27.36	2.23	540.86	270.86	22.26

(d)

Fig. 4. Average performance over all runs of five generated algorithms on the test instance of each problem. The five algorithms are inferred from ALDes's five training trials with different seeds. In each subfigure, the *i*th row reports the test results after training on the first *i* problems. For example, the (3, 2)th cell of subfigure (a) shows the performance of algorithms for F2 (148.07), in which algorithms are inferred from the ALDes that has been trained on F1–F3. (a) Test results with EWC on continual task 1. (b) Test results without EWC on continual task 1. (c) Test results with EWC on continual task 2. (d) Test results without EWC on continual task 2.

of attraction) and prior problems. Nonetheless, the training converges to a comparable performance with the one-off curve. Overall, results of Figs. 4 and 5 suggest that with EWC, ALDes is able to accumulate knowledge learned from prior tasks. The knowledge enables ALDes to fast adapt to future designs. Even in cases of significant concept drift, ALDes exhibits eligible convergence. All these showcase ALDes's potential for continual design of algorithms for open-ended problem solving.

D. Application to Beamforming in RIS-Aided Communications

1) *Problem Description*: The RIS is a cutting-edge technology designed to improve cost-effective communications [68]. Functioning as a planar passive radio structure, it features reconfigurable passive elements that can independently adjust the phase shift of incoming signals. By working together, these elements generate a directional beam, enhancing signal quality.

TABLE II
PERFORMANCE (MEAN $\pm$ STD) OVER ALL RUNS ON TEST INSTANCES OF THE BEAMFORMING PROBLEM. Alg_{beam}^* 'S PERFORMANCE IS AVERAGED OVER FIVE ALGORITHMS INFERRED FROM ALDES'S FIVE TRAINING TRIALS WITH DIFFERENT SEEDS. BEST RESULTS ACCORDING TO PAIR-WISE WILCOXON SIGN TEST (5% SIGNIFICANCE LEVEL) ARE WITH GREY BACKGROUND

	Number of RIS elements in the test instances				
	120	160	280	320	400
Random	0.0442 $\pm$ 7.94E-04	0.0425 $\pm$ 6.56E-04	0.0402 $\pm$ 8.30E-04	0.0390 $\pm$ 6.67E-04	0.0375 $\pm$ 1.88E-04
Sequential	0.0382 $\pm$ 6.19E-04	0.0387 $\pm$ 6.75E-04	0.0374 $\pm$ 4.17E-04	0.0369 $\pm$ 4.38E-04	0.0354 $\pm$ 8.27E-04
ILS	0.0333 $\pm$ 3.74E-04	0.0314 $\pm$ 2.49E-04	0.0285 $\pm$ 1.19E-04	0.0279 $\pm$ 1.82E-04	0.0278 $\pm$ 1.15E-04
SA	0.0392 $\pm$ 8.13E-04	0.0381 $\pm$ 4.55E-04	0.0357 $\pm$ 1.89E-04	0.0349 $\pm$ 3.52E-04	0.0346 $\pm$ 6.17E-04
GA	0.0359 $\pm$ 7.24E-04	0.0334 $\pm$ 1.13E-04	0.0302 $\pm$ 2.21E-04	0.0316 $\pm$ 1.67E-04	0.0309 $\pm$ 5.67E-04
Alg_{beam}^*	0.0331 $\pm$ 4.03E-07	0.0312 $\pm$ 1.08E-07	0.0281 $\pm$ 1.23E-07	0.0272 $\pm$ 2.39E-08	0.0259 $\pm$ 4.41E-08

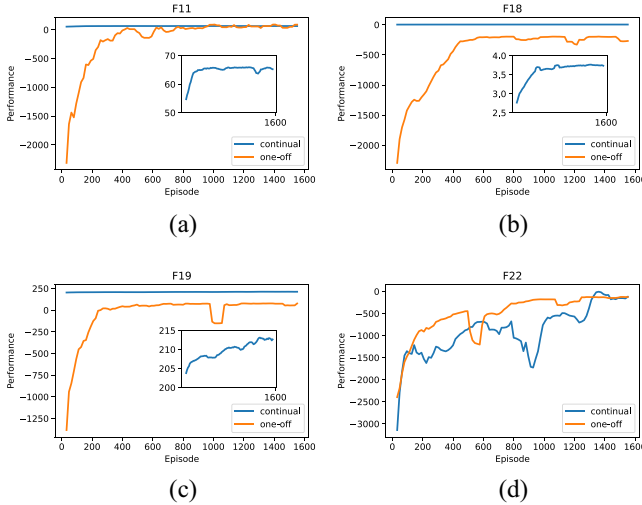


Fig. 5. ALDes's training performance versus episodes. Averaged over five trials. The one-off curves report the results of training from scratch. The continual curves report the results of training under the continual task settings. The insets resize the continual curves to show the convergent tendency. (a) F11. (b) F18. (c) F19. (d) F22.

Specifically, we consider the RIS-aided downlink multiuser MISO system detailed in [69]. In this setup, a BS with multiple antennas serves K single-antenna users via an RIS with N elements, facilitating nonline-of-sight links. The aim is to maximize the sum rate of all users while adhering to transmit power constraints, achieved through joint optimization of BS beamforming and RIS phase shifts [69]

$$\max_{\mathbf{w}_k, \Theta} \sum_{k=1}^K \log_2 \left(1 + \frac{|\left(\mathbf{h}_{d,k}^H + \mathbf{h}_{r,k}^H \Theta \mathbf{G}\right) \mathbf{w}_k|^2}{\sum_{j \neq k}^K |\left(\mathbf{h}_{d,k}^H + \mathbf{h}_{r,k}^H \Theta \mathbf{G}\right) \mathbf{w}_j|^2 + \sigma^2} \right) \quad (15a)$$

$$\text{s.t. } \theta_n = \beta_n e^{j\phi_n} \quad (15b)$$

$$\phi_n = \frac{\tau_n 2\pi}{2^b}, \tau_n \in \{0, \dots, 2^b - 1\} \quad (15c)$$

$$\sum_{k=1}^K \|\mathbf{w}_k\|^2 \leq P_T \quad (15d)$$

where $\mathbf{w}_k \in \mathbb{C}^{M \times 1}$ is the active beamforming at the BS toward user k ; $\Theta = \text{diag}(\theta_1, \dots, \theta_N, \dots, \theta_N)$ is a diagonal matrix with RIS phase-shifts being the diagonal values; (15c) represents that RIS phase shifts take b -bit values; $\mathbf{h}_{d,k} \in \mathbb{C}^{M \times 1}$, $\mathbf{G} \in$

$\mathbb{C}^{N \times M}$, and $\mathbf{h}_{r,k} \in \mathbb{C}^{N \times 1}$ are the channels BS-user k , BS-RIS, and RIS-user k , respectively, which are modeled as random matrices; $\beta_n = 1$ is for all RIS elements; and (15d) restricts the transmit power being not larger than P_T .

The problem is nonconvex and NP-hard. Additionally, landscape analysis indicates rugged and scattered local optimums, particularly with large-scale RIS elements [69]. The typical solver employs water-filling [70] for BS beamforming and individual estimation of RIS element phase shifts. However, decoupled estimation is deemed inadequate [69]. Metaheuristics offer potential for globally searching the rugged and highly coupled space.

2) *Applying ALDes*: We apply the proposed ALDes to generate metaheuristic algorithms for estimating RIS phase shifts. Target problem instances differ in the number of RIS elements. Baselines include random beamforming, sequential beamforming [71], ILS, SA, and GA.⁹ Table II reports the results on test instances, in which Alg_{beam}^* is the algorithm generated by ALDes. All metaheuristic peers handle the constraints by penalizing solutions' fitness with constraint violations. The results measure the quality of service for all users; smaller values refer to better service quality.

Alg_{beam}^* obtains superior performance on all instances. It is a GA-style algorithm (Algorithm 5, with its sequence representation shown in Fig. S2(e) of the Supplementary Material). Its difference from the baseline GA is the 1d reset and greedy selection. The difference enables Alg_{beam}^* obtaining much better performance. A possible explanation for the performance is that the population-based stable and greedy phase shift of the homogeneous RIS elements carefully locates the scattered optimums in which the global one may exist. The explanation also suits ILS, which obtains the second-best results on most instances. In comparison, the representative sequential beamforming and other baselines underperform. All these demonstrate ALDes's efficiency.

E. Application to Power System Restoration

1) *Problem Description*: Power system restoration involves scheduling BS and NBS units to re-energize the power network and loads after a complete blackout, ensuring the system

⁹Sequential beamforming exhaustively adjusts the phase shift of each element individually while maintaining the remaining phase shifts unchanged. GA consists of tournament mating selection, one-point crossover, random mutation of probability 0.2, and round-Robin environmental selection, following [69]. ILS, SA, and other settings are the same as Section IV-A.

Algorithm 5: Alg_{beam}^* for Beamforming

Input: initial solutions S
1 while *stopping criterion not met* **do**
 2 $S = \text{tournament}(S)$
 3 $S_{\text{new}} = \text{cross_n}(S, n = 1)$
 4 $S_{\text{new}} = \text{reset_n}(S_{\text{new}}, n = 1)$
 5 $S = \text{greedy_select}(S, S_{\text{new}})$
Output: optimal phase shifts from S

safety and stability. The restoration process consists of BS units restart followed by NBS units reboot, receiving cranking power from BS units. Critical loads are then re-energized to maintain system frequency and voltage. Subsequently, remaining loads are re-energized based on priority. Decision variables include restoration sequence and path for BS units, NBS units, and loads, aiming to minimize the total restoration time

$$\min_{\mathbf{v}_a, \mathbf{h}_b} \sum_{a=1}^A \mathbf{v}_a^T \mathbf{w}_a + \sum_{b=1}^B \mathbf{h}_b^T \mathbf{l}_b \quad (16a)$$

$$\text{s.t.} \quad \sum_{c=1}^C v_{ca} = 1, a = 1, 2, \dots, A \quad (16b)$$

$$\sum_{a=1}^A h_{ab} = 1, b = 1, 2, \dots, B \quad (16c)$$

$$0 \leq P_i^{\text{gen}} \leq P_i^{\text{gen}, \max} \quad \forall i \in \Omega_{\text{Gen}} \quad (16d)$$

$$Q_i^{\text{gen}, \min} \leq Q_i^{\text{gen}} \leq Q_i^{\text{gen}, \max} \quad \forall i \in \Omega_{\text{Gen}} \quad (16e)$$

$$0 \leq P_{a,t-1}^{\text{gen}} - P_{a,t}^{\text{gen}} \leq R_G \Delta t \quad \forall a \in \Omega_{\text{NBS}} \quad (16f)$$

$$P_{m,t} = \sum_{mn \in \Omega_{Br}} P_{mn,t} \quad (16g)$$

$$Q_{m,t} = \sum_{mn \in \Omega_{Br}} Q_{mn,t} \quad (16h)$$

$$P_{mn}^2 + Q_{mn}^2 \leq S_{mn}^2 z_{mn} \quad \forall mn \in \Omega_{Br} \quad (16i)$$

$$U_m^{\min} \leq U_m \leq U_m^{\max} \quad \forall m \in \Omega_{\text{Bus}} \quad (16j)$$

$$\sum_{t=1}^T (1 - z_{mc,t}) \geq t_c^{\text{start}} \quad \forall mc \in \Omega_{Br_BS} \quad \forall c \in \Omega_{BS} \quad (16k)$$

$$z_{ma,t} \leq z_{a,t} \leq \sum_{ma \in \Omega_{Br_NBS}} z_{ma,t} \quad \forall a \in \Omega_{\text{NBS}} \quad (16l)$$

where binary vectors $\mathbf{v}_a = (v_{1a}, \dots, v_{ca}, \dots, v_{Ca})$, $a = 1, 2, \dots, A$, and $\mathbf{h}_b = (h_{1b}, \dots, h_{ab}, \dots, h_{Ab})$, $b = 1, 2, \dots, B$, are the decision variables that determine the connectivity from BS units to NBS units and that from NBS units to loads, respectively; A , B , and C are the numbers of NBS units, loads, and BS units, respectively; $\mathbf{v}_a$ donates which BS unit supplies cranking power to NBS unit a ; $\mathbf{h}_b$ determines which NBS unit supplies electricity to load b ; $\mathbf{w}_a$ and $\mathbf{l}_b$ stand for the distances of the shortest time path from BS units to NBS units and that from NBS units to loads, respectively; P_i^{gen} , $P_{m,t}$, and P_{mn} are the active power of generator i , bus m , and branch mn , respectively; Q_i^{gen} , $Q_{m,t}$, and Q_{mn} are the reactive power of generator i , bus m , and branch mn , respectively; $P_i^{\text{gen}, \max}$ is the maximum active power of generator i ; $Q_i^{\text{gen}, \min}$

TABLE III

PERFORMANCE (MEAN $\pm$ STD) ON THE TEST INSTANCE OF THE POWER SYSTEM RESTORATION PROBLEM. Alg_{restor}^* 'S PERFORMANCE IS AVERAGED OVER FIVE ALGORITHMS INFERRED FROM ALDES'S FIVE TRAINING TRIALS WITH DIFFERENT SEEDS. BEST RESULTS ACCORDING TO PAIR-WISE WILCOXON SIGN TEST (5% SIGNIFICANCE LEVEL) ARE WITH GREY BACKGROUND

Algorithms	Restoration Time
CPLEX	3697.82 $\pm$ 0.00E+00
ILS	3268.51 $\pm$ 1.43E+03
SA	3379.51 $\pm$ 3.59E+03
GA	3105.23 $\pm$ 1.07E+02
Alg_{restor}^*	3016.52 $\pm$ 2.24E+01

Algorithm 6: Alg_{restor}^* for Power System Restoration

Input: initial solutions S
1 while *stopping criterion not met* **do**
 2 $S = \text{traverse}(S)$
 3 $S_{\text{new}} = \text{cross_uniform}(S, p = 0.2)$
 4 $S_{\text{new}} = \text{reset_n}(S_{\text{new}}, n = 1)$
 5 $S = \text{pairwise_select}(S, S_{\text{new}})$
Output: best solution from S

and $Q_i^{\text{gen}, \max}$ are the minimum and maximum reactive power of generator i , respectively; R_G is generators' ramp rate; S_{mn} is the apparent power of branch mn ; U_m is the voltage of bus m ; $U_m^{\min}$ and $U_m^{\max}$ are the minimum and maximum voltage of bus m , respectively; t_c^{start} is the time that BS unit c starts generating power; z_{mn} , $z_{mc,t}$, $z_{ma,t}$, and $z_{a,t}$ are the restore status of branch mn , that of branch mc that directly connects to BS unit c at time t , that of branch ma that directly connects to NBS unit a at time t , and that of NBS unit a at time t , respectively.

Equation (16) represents a binary integer programming problem with highly nonconvex constraints, typically NP-hard. Attaining a feasible solution satisfying these constraints using conventional analytical solvers is challenging and computationally intensive as the power system scale increases. Metaheuristics offer a potential solution.

2) *Applying ALDes:* We apply ALDes to generate metaheuristic algorithms for the power system restoration problem. Two instances of the IEEE 39-bus system are implemented. They differ in the locations of BS and NBS units. Baselines include the analytical solver CPLEX, ILS, SA, and GA. Table III reports the results on the test instance, in which Alg_{restor}^* is the algorithm generated by ALDes. All metaheuristic peers handle the constraints by penalizing solutions' fitness with constraint violations. The results measure the restoration time; smaller values refer to shorter restoration time.

Alg_{restor}^* obtains much better result than baselines. It performs population-based local search (Algorithm 6, with its sequence representation shown in Fig. S2(f) of the Supplementary Material). That is, every current solution is chosen to search from (*traverse*), resulting in a new solution; the current solution is paired with the new one, and the better between them is selected (*pairwise_select*). The population of local search pathways are interconnected through uniform crossover. The key difference of Alg_{restor}^* from metaheuristic baselines is its diversity maintenance by many local search pathways, enabling it to explore feasible

and promising solutions within the fragmented solution space. CPLEX lags behind metaheuristic solvers, because it smooths the fragmented space by relaxation, potentially missing the true optimal. All these demonstrate metaheuristics' significance in practical problem solving and ALDes's ability to customize metaheuristic algorithms.

V. CONCLUSION

This article has presented the ALDes, for automated design of metaheuristic algorithms. ALDes formulates metaheuristic algorithm design as a sequence generation task and proposes a sequence representation of metaheuristic algorithms. The new formulation and representation push the boundary of automated algorithm design to a new level: designing algorithms with diverse structures that can fully discover potentials over the metaheuristic family. ALDes leverages an autoregressive generative network with a transformer backbone to handle the task. The network is trained offline to infer high-performance algorithms to subsequently solve many problem instances from the target problem domain. Extensive experiments have revealed that ALDes can customize various types of algorithms (e.g., VLS-, ILS-, and GA-style) and even unconventional ones for different problem-solving contexts. The generated algorithms have suited well corresponding target problems and significantly outperformed human-crafted baselines on 24 out of 25 problems.

This article has implemented ALDes with a straightforward model architecture, highlighting the efficiency and promise of the autoregressive generation idea. Future research is worth investigating more complex and larger models to develop a *large pretrained designer*, which could advance the long pursuit of general, open-ended, and automated problem solving. Furthermore, this article has empirically verified the proposed ideas by comparing with fine-tuned baseline algorithms. While the proposed method has reached optimal on 6 of the 23 problems and got very close or even better results concerning the target values reported in [61], this article does not claim state-of-the-art performance; a comprehensive comparison with other automated algorithm design methods is an important future step. In addition, this article used precollected landscape features as factors to embed target problems. But the significance of features for discriminating different problems is different [72], [73]. Empirical feature selection prior to algorithm design or new embedding methods independent of landscape features are worth studying in the future.

REFERENCES

- [1] F. W. Glover and G. A. Kochenberger, *Handbook of Metaheuristics*. New York, NY, USA: Springer, 2006.
- [2] J. Swan et al., "Metaheuristics 'in the large,'" *Eur. J. Oper. Res.*, vol. 297, no. 2, pp. 393–406, Mar. 2022.
- [3] Q. Zhao, Q. Duan, B. Yan, S. Cheng, and Y. Shi, "Automated design of metaheuristic algorithms: A survey," *Trans. Mach. Learn. Res.*, pp. 1–30, Feb. 2024. [Online]. Available: <https://openreview.net/forum?id=qhtHsvF5zj>
- [4] T. Stützle and M. López-Ibáñez, "Automated design of metaheuristic algorithms," in *Handbook of Metaheuristics*. New York, NY, USA: Springer, 2019, pp. 541–579.
- [5] R. Qu, G. Kendall, and N. Pillay, "The general combinatorial optimization problem: Towards automated algorithm design," *IEEE Comput. Intell. Mag.*, vol. 15, no. 2, pp. 14–23, May 2020.
- [6] P. Kerschke, H. H. Hoos, F. Neumann, and H. Trautmann, "Automated algorithm selection: Survey and perspectives," *Evol. Comput.*, vol. 27, no. 1, pp. 3–45, Mar. 2019.
- [7] E. Schede et al., "A survey of methods for automated algorithm configuration," *J. Artif. Intell. Res.*, vol. 75, pp. 425–487, Oct. 2022.
- [8] F. Hutter, H. H. Hoos, and K. Leyton-Brown, "Sequential model-based optimization for general algorithm configuration," in *Proc. 5th Int. Conf. Learn. Intell. Optim.*, 2011, pp. 507–523.
- [9] M. Lindauer et al., "SMAC3: A versatile Bayesian optimization package for hyperparameter optimization," *J. Mach. Learn. Res.*, vol. 23, no. 54, pp. 1–9, Feb. 2022.
- [10] M. López-Ibáñez, J. Dubois-Lacoste, L. P. Cáceres, M. Birattari, and T. Stützle, "The irace package: Iterated racing for automatic algorithm configuration," *Oper. Res. Perspect.*, vol. 3, pp. 43–58, Sep. 2016.
- [11] F. Hutter, H. H. Hoos, K. Leyton-Brown, and T. Stützle, "ParamILS: An automatic algorithm configuration framework," *J. Artif. Intell. Res.*, vol. 36, pp. 267–306, Oct. 2009.
- [12] E. K. Burke et al., "Hyper-heuristics: A survey of the state of the art," *J. Oper. Res. Soc.*, vol. 64, no. 12, pp. 1695–1724, Jul. 2013.
- [13] N. Pillay and R. Qu, *Hyper-Heuristics: Theory and Applications*. New York, NY, USA: Springer, 2018.
- [14] J. R. Koza, "Genetic programming as a means for programming computers by natural selection," *Statist. Comput.*, vol. 4, no. 2, pp. 87–112, Jun. 1994.
- [15] T. Bäck and H.-P. Schwefel, "An overview of evolutionary algorithms for parameter optimization," *Evol. Comput.*, vol. 1, no. 1, pp. 1–23, Mar. 1993.
- [16] L. C. T. Bezerra, M. López-Ibáñez, and T. Stützle, "Automatic component-wise design of multiobjective evolutionary algorithms," *IEEE Trans. Evol. Comput.*, vol. 20, no. 3, pp. 403–417, Jun. 2016.
- [17] A. Blot, M.-É. Kessaci, L. Jourdan, and H. H. Hoos, "Automatic configuration of multi-objective local search algorithms for permutation problems," *Evol. Comput.*, vol. 27, no. 1, pp. 147–171, Mar. 2019.
- [18] L. C. T. Bezerra, L. Manuel, and S. Thomas, "Automatically designing state-of-the-art multi-and many-objective evolutionary algorithms," *Evol. Comput.*, vol. 28, no. 2, pp. 195–226, Jun. 2020.
- [19] C. L. Camacho-Villalón, M. Dorigo, and T. Stützle, "PSO-X: A component-based framework for the automatic design of particle swarm optimization algorithms," *IEEE Trans. Evol. Comput.*, vol. 26, no. 2, pp. 402–416, Jun. 2022.
- [20] W. Yi, R. Qu, L. Jiao, and B. Niu, "Automated design of metaheuristics using reinforcement learning within a novel general search framework," *IEEE Trans. Evol. Comput.*, vol. 27, no. 4, pp. 1072–1084, Aug. 2023.
- [21] Y. Tian, X. Li, H. Ma, X. Zhang, K. C. Tan, and Y. Jin, "Deep reinforcement learning based adaptive operator selection for evolutionary multi-objective optimization," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 7, no. 4, pp. 1051–1064, Aug. 2023.
- [22] F. Ye, C. Doerr, H. Wang, and T. Bäck, "Automated configuration of genetic algorithms by tuning for anytime performance," *IEEE Trans. Evol. Comput.*, vol. 26, no. 6, pp. 1526–1538, Dec. 2022.
- [23] S. N. Richter and D. R. Tauritz, "The automated design of probabilistic selection methods for evolutionary algorithms," in *Proc. Genet. Evol. Comput. Conf. Companion*, 2018, pp. 1545–1552.
- [24] S. N. Richter, M. G. Schoen, and D. R. Tauritz, "Evolving mean-update selection methods for CMA-ES," in *Proc. Genet. Evol. Comput. Conf. Companion*, 2019, pp. 1513–1517.
- [25] J. R. Woodward and R. Bai, "Why evolution is not a good paradigm for program induction: A critique of genetic programming," in *Proc. 1st ACM/SIGEVO Summit Genet. Evol. Comput.*, 2009, pp. 593–600.
- [26] E. Smorodkina and D. Tauritz, "Toward automating EA configuration: The parent selection stage," in *Proc. IEEE Congr. Evol. Comput.*, 2007, pp. 63–70.
- [27] R. Rivers and D. R. Tauritz, "Evolving black-box search algorithms employing genetic programming," in *Proc. 15th Annu. Conf. Companion Genet. Evol. Comput.*, 2013, pp. 1497–1504.
- [28] J. F. Miller, *Cartesian Genetic Programming*. New York, NY, USA: Springer, 2011, pp. 17–34.
- [29] W. Kantschik, P. Dittrich, M. Brameier, and W. Banzhaf, "Meta-evolution in graph GP," in *Proc. 2nd Eur. Conf. Genet. Programm.*, 1999, pp. 15–28.

- [30] S. Shirakawa and T. Nagao, "Evolution of search algorithms using graph structured program evolution," in *Proc. 12th Eur. Conf. Genet. Programm.*, 2009, pp. 109–120.
- [31] P. Ryser-Welch, J. F. Miller, J. Swan, and M. A. Trefzer, "Iterative cartesian genetic programming: Creating general algorithms for solving travelling salesman problems," in *Proc. 19th Eur. Conf. Genet. Programm.*, 2016, pp. 294–310.
- [32] S. Kirkpatrick, C. D. Gelatt Jr., and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, May 1983.
- [33] Y. Shi and R. Eberhart, "A modified particle swarm optimizer," in *Proc. IEEE Int. Conf. Evol. Comput.*, 1998, pp. 69–73.
- [34] J. H. Holland, "Genetic algorithms and the optimal allocation of trials," *SIAM J. Comput.*, vol. 2, no. 2, pp. 88–105, Jun. 1973.
- [35] A. Franzin and T. Stützle, "Revisiting simulated annealing: A component-based analysis," *Comput. Oper. Res.*, vol. 104, pp. 191–206, Apr. 2019.
- [36] Y. Sakurai, K. Takada, T. Kawabe, and S. Tsuruta, "A method to control parameters of evolutionary algorithms by using reinforcement learning," in *Proc. 6th Int. Conf. Signal-Image Technol. Internet Based Syst.*, 2010, pp. 74–79.
- [37] A. Buzdalova, V. Kononov, and M. Buzdalov, "Selecting evolutionary operators using reinforcement learning: Initial explorations," in *Proc. Annu. Conf. Genet. Evol. Comput.*, 2014, pp. 1033–1036.
- [38] S. Adriaensen and A. Nowé, "Towards a white box approach to automated algorithm design," in *Proc. 25th Int. Joint Conf. Artif. Intell.*, 2016, pp. 554–560.
- [39] W. Meng and R. Qu, "Automated design of search algorithms: Learning on algorithmic components," *Expert Syst. Appl.*, vol. 185, Dec. 2021, Art. no. 115493.
- [40] W. Yi, R. Qu, and L. Jiao, "Automated algorithm design using proximal policy optimisation with identified features," *Expert Syst. Appl.*, vol. 216, Apr. 2023, Art. no. 119461.
- [41] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015.
- [42] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [43] A. Vaswani et al., "Attention is all you need," in *Proc. 31st Adv. Neural Inf. Process. Syst.*, 2017, pp. 1–11.
- [44] A. P. Parikh, O. Täckström, D. Das, and J. Uszkoreit, "A decomposable attention model for natural language inference," 2016, *arXiv:1606.01933*.
- [45] Z. Lin et al., "A structured self-attentive sentence embedding," 2017, *arXiv:1703.03130*.
- [46] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [47] B. Zoph and Q. V. Le, "Neural architecture search with reinforcement learning," in *Proc. Int. Conf. Learn. Represent.*, 2017, pp. 1–16.
- [48] B. Zoph, V. Vasudevan, J. Shlens, and Q. V. Le, "Learning transferable architectures for scalable image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 8697–8710.
- [49] M. Popova, O. Isayev, and A. Tropsha, "Deep reinforcement learning for de novo drug design," *Sci. Adv.*, vol. 4, no. 7, 2018, Art. no. eaap7885.
- [50] D. Shin et al., "Deep reinforcement learning-designed radiofrequency waveform in MRI," *Nat. Mach. Intell.*, vol. 3, no. 11, pp. 985–994, Nov. 2021.
- [51] R. Loynd, R. Fernandez, A. Celikyilmaz, A. Swaminathan, and M. Hausknecht, "Working memory graphs," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 6404–6414.
- [52] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, "Trust region policy optimization," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 1889–1897.
- [53] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," 2013, *arXiv:1301.3781*.
- [54] A. Scarlatos and A. Lan, "Tree-based representation and generation of natural and mathematical language," in *Proc. 61st Annu. Meeting Assoc. Comput. Linguist.*, 2023, pp. 3714–3730.
- [55] O. Mersmann, B. Bischl, H. Trautmann, M. Preuss, C. Weihs, and G. Rudolph, "Exploratory landscape analysis," in *Proc. 13th Annu. Conf. Genet. Evol. Comput.*, 2011, pp. 829–836.
- [56] I. M. Sobol', "On the distribution of points in a cube and the approximate evaluation of integrals," *Zhurnal Vychislitel'noi Matematiki i Matematicheskoi Fiziki*, vol. 7, no. 4, pp. 86–112, 1967.
- [57] Q. Renau, C. Doerr, J. Dreio, and B. Doerr, "Exploratory landscape analysis is strongly sensitive to the sampling strategy," in *Proc. 16th Parallel Problem Solving Nat.*, 2020, pp. 139–153.
- [58] K. M. Malan and A. P. Engelbrecht, "A survey of techniques for characterising fitness landscapes and some possible ways forward," *Inf. Sci.*, vol. 241, pp. 148–163, Aug. 2013.
- [59] J. Kirkpatrick et al., "Overcoming catastrophic forgetting in neural networks," *Proc. Nat. Acad. Sci.*, vol. 114, no. 13, pp. 3521–3526, Mar. 2017.
- [60] M. De Lange et al., "A continual learning survey: Defying forgetting in classification tasks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 7, pp. 3366–3385, Jul. 2022.
- [61] C. Doerr, F. Ye, N. Horesh, H. Wang, O. M. Shir, and T. Bäck, "Benchmarking discrete optimization heuristics with IOHprofiler," *Appl. Soft Comput.*, vol. 88, Mar. 2020, Art. no. 106027.
- [62] H. R. Lourenço, O. C. Martin, and T. Stützle, "Iterated local search," in *Handbook of Metaheuristics*. New York, NY, USA: Springer, 2003, pp. 320–353.
- [63] F. Glover, "Tabu search—Part I," *ORSA J. Comput.*, vol. 1, no. 3, pp. 190–206, Aug. 1989.
- [64] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2017, *arXiv:1412.6980*.
- [65] N. Mladenović and P. Hansen, "Variable neighborhood search," *Comput. Oper. Res.*, vol. 24, no. 11, pp. 1097–1100, Nov. 1997.
- [66] A. Lucas, "Ising formulations of many NP problems," *Front. Phys.*, vol. 2, Feb. 2014, Art. no. 74887.
- [67] Y.-Q. Chen, Y. Chen, C.-K. Lee, S. Zhang, and C.-Y. Hsieh, "Optimizing quantum annealing schedules with monte carlo tree search enhanced with neural networks," *Nat. Mach. Intell.*, vol. 4, no. 3, pp. 269–278, 2022.
- [68] J. Yuan, Y.-C. Liang, J. Joung, G. Feng, and E. G. Larsson, "Intelligent reflecting surface-assisted cognitive radio system," *IEEE Trans. Commun.*, vol. 69, no. 1, pp. 675–687, Jan. 2021.
- [69] B. Yan, Q. Zhao, M. Li, J. Zhang, J. A. Zhang, and X. Yao, "Fitness landscape analysis and niching genetic approach for hybrid beamforming in RIS-aided communications," *Appl. Soft Comput.*, vol. 131, Dec. 2022, Art. no. 109725.
- [70] W. Yu, W. Rhee, S. Boyd, and J. M. Cioffi, "Iterative water-filling for Gaussian vector multiple-access channels," *IEEE Trans. Inf. Theory*, vol. 50, no. 1, pp. 145–152, Jan. 2004.
- [71] B. Di, H. Zhang, L. Song, Y. Li, Z. Han, and H. V. Poor, "Hybrid beamforming for reconfigurable intelligent surface based multi-user communications: Achievable rates with limited discrete phase shifts," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 8, pp. 1809–1822, Aug. 2020.
- [72] Q. Renau, J. Dreio, C. Doerr, and B. Doerr, "Expressiveness and robustness of landscape features," in *Proc. Genet. Evol. Comput. Conf. Companion*, 2019, pp. 2048–2051.
- [73] Q. Renau, J. Dreio, C. Doerr, and B. Doerr, "Towards explainable exploratory landscape analysis: Extreme feature selection for classifying BBOb functions," in *Proc. 24th Int. Conf. Appl. Evol. Comput.*, 2021, pp. 17–33.



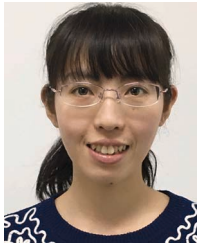
Qi Zhao received the Ph.D. degree in management science and engineering from Beijing University of Technology, Beijing, China, in 2019.

He was a joint Ph.D. student in computer science with the University of New South Wales, Sydney, NSW, Australia, from 2017 to 2018. He is a Research Assistant Professor with the Department of Computer Science and Engineering, Southern University of Science and Technology, Shenzhen, China. His research interests include automated machine learning, operations research, and evolutionary computation.



Tengfei Liu received the B.E. degree in artificial intelligence and automation from Huazhong University of Science and Technology, Wuhan, China, in 2021. He is currently pursuing the M.E. degree in computer science with the Department of Computer Science and Engineering, Southern University of Science and Technology, Shenzhen, China.

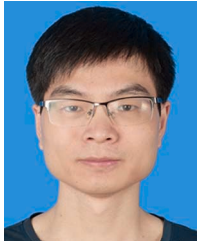
His research interests include evolutionary computation, automated machine learning, and reinforcement learning.



Bai Yan received the Ph.D. degree in optical engineering from Beijing University of Technology, Beijing, China, in 2019.

She was a joint Ph.D. student in computational and communication engineering with the University of Technology Sydney, Sydney, NSW, Australia, from 2017 to 2018. She is currently a Research Assistant Professor with the Research Institute of Trustworthy Autonomous Systems, Southern University of Science and Technology, Shenzhen, China. Her research interests include computational

intelligence, optimization of wireless networks, and reconfigurable intelligent surface.



Qiqi Duan is currently pursuing the Ph.D. degree with Harbin Institute of Technology, Harbin, China, while studying with the Southern University of Science and Technology, Shenzhen, China.

He is one of the developers of the open-source Python library PyPop7 for evolutionary algorithms. His interests are evolutionary computation, metalearning/multilevel evolution, swarm intelligence, and distributed systems.

Dr. Duan obtained one Best Paper nomination on PPSN-2022.



Jian Yang received the B.S. and M.S. degrees from the Department of Control Science and Engineering, Harbin Institute of Technology (HIT), Harbin, China, in 2005 and 2007, respectively, and the Ph.D. degree from the Department of Mechanical Engineering, HIT (Shenzhen), Shenzhen, China, in 2019.

He is currently a Research Associate Professor with the Department of Computer Science and Engineering, Southern University of Science and Technology, Shenzhen. His research interests include

computational intelligence, swarm intelligence, and intelligent robotics.



Yuhui Shi (Fellow, IEEE) received the Ph.D. degree in electronic engineering from Southeast University, Nanjing, China, in 1992.

He is currently a Chair Professor with the Department of Computer Science and Engineering, Southern University of Science and Technology, Shenzhen, China. He has co-authored the book *Swarm Intelligence* with Dr. Kennedy and Prof. Eberhart and another book *Computational Intelligence: Concept to Implementation* with Prof. Eberhart. His main research interests are

artificial intelligence, particularly swarm intelligence, and their applications.

Dr. Shi is the Editor-in-Chief of *International Journal of Swarm Intelligence Research*.